DATA SCIENCE AND MACHINE LEARNING · END-SEMESTER PROJECT REPORT

AlphaEdge

Ranking Indian Stocks with Machine Learning

Collecting and cleaning six years of market data, testing eight model versions, and building a system that picks stocks by itself every trading day

Submitted by

Uzair Manjre Roll number: [Roll number]

Pal Gandhi Roll number: [Roll number]

Malav Thakor Roll number: [Roll number]

Shlok Barot Roll number: [Roll number]

Under the guidance of

[Guide name]

[Department name]

[Institute name]

November 2026

Code and data: github.com/UzairManjre/stock_predict

Abstract

This project asks whether a program that uses only public data can choose, each trading day, a group of Indian stocks that will grow more than the market over the following month. We built **AlphaEdge**, a complete system that does this for 209 companies listed on the National Stock Exchange of India (NSE), using the NIFTY 50 index as "the market".

We collected six years of daily prices (312,064 rows, August 2020 to October 2026) and 169,343 company announcements into a PostgreSQL database, wrote scripts that repair errors in the raw data, and built a table of 70 columns per stock per day describing trend, risk, trading activity, sector and the tone of announcements (scored with the FinBERT language model). We then trained and compared a deep neural network for time series (the Temporal Fusion Transformer), gradient-boosted decision trees (LightGBM), a model that learns the order of stocks rather than their prices, and simple rules such as momentum. Every model was tested the same way: walk-forward, on 18 months of data it had never seen.

The final model averages the rankings of four models. On the 18 test months its forecast ranges held the real outcome 80–83% of the time, as intended, and its 20 top-ranked stocks grew more than the NIFTY 50 over the next month on 68% of days, rising to 74% on days a built-in confidence flag marked as high confidence. Against the stricter benchmark of the average stock, where 50% means no skill, the picks won on 65% of days (70% on high-confidence days). The system now runs by itself after every trading day, publishes its picks on a dashboard and checks its own accuracy as results arrive.

Keywords: stock ranking, machine learning, gradient boosting, Temporal Fusion Transformer, walk-forward testing, ensemble, NSE

Contents

Abstract **2**
1. Introduction **4**
1.1 The problem 4
1.2 What we built 4
1.3 Main results 5
2. Key Ideas in Plain Words **5**
2.1 Why stock prediction is hard 5
3. Data Collection **6**
3.1 Storage 6
4. Data Cleaning **7**
4.1 The price repair script 7
4.2 Preventing leaks from the future 8
5. Processing the Data: Feature Engineering **8**
5.1 Announcement tone with FinBERT 9
5.2 Comparing, not measuring 9
5.3 What we predict 9
6. The Models We Trained **9**
6.1 Temporal Fusion Transformer 10
6.2 Gradient-boosted trees (LightGBM) 11
6.3 Rank regression 11
6.4 Momentum and per-stock history 11
6.5 The final model: a blend of four 12
6.6 The confidence flag 12
6.7 What did not make it 12
7. How We Train and Test **12**
7.1 Walk-forward testing 12
7.2 How we measure success 13
7.3 Rules that keep us honest 13
8. The Development Journey: Eight Iterations **14**
9. Results **15**
9.1 Forecast ranges and ranking quality 15
9.2 Hit rates and the confidence flag 15
9.3 Backtest 16
9.4 What announcements tell us 17
10. How the System Works Day to Day **17**
11. Limitations and Future Work **18**
12. Conclusion **19**
References **19**
Appendix A. Running the Project **20**

1. Introduction

1.1 The problem

Millions of people in India invest in the stock market, and the question they face most is which companies to buy. Over long periods, most professional fund managers do not beat the market index. We wanted to know whether a program using only public data (past prices and company announcements) can do measurably better than chance. We framed the question so that it can be tested:

> **The question**
>
> Each trading day, can the system choose 20 stocks out of 209 such that, as a group, they grow more than the NIFTY 50 index over the next 21 trading days (about one month), more often than chance would allow?

This framing matters. We first tried to predict each stock's exact price change and found that no model could do that better than a coin toss. Changing the question to "which stocks will do better than the others?" is what made the problem learnable (Chapter 8).

1.2 What we built

AlphaEdge is a complete pipeline from raw data to a daily list of picks (Figure 1). Scripts collect prices and announcements, repair them in a PostgreSQL database, and turn them into features. Several kinds of model are trained and compared on an honest walk-forward test, and the best design runs automatically after every trading day, with a dashboard and a record of how its past picks actually performed.



Figure 1. The AlphaEdge pipeline. Each box is a set of Python scripts in the project repository.

1.3 Main results

Table 1. Headline results of the final model on the 18 test months (late March 2025 to September 2026).

Measure	Result	What it means
Forecast range accuracy	83% (5-day), 80% (21-day)	Real outcomes fell inside our predicted 10th–90th percentile range as often as an honest forecast should (80%).
Top 20 beat the NIFTY 50	68% of all days; 74% of high-confidence days	Share of days on which our 20 picks grew more than the index over the next month. High-confidence days are 48% of days.
Top 20 beat the average stock	65% of all days; 70% of high-confidence days	The stricter test: 50% would mean no skill.
Direction of a single stock	50–52%	Calling whether one stock goes up or down is close to a coin toss, which is why we rank instead.

2. Key Ideas in Plain Words

No background in finance is needed for this report. These are the terms we use.

Table 2. Terms used in this report.

Term	Meaning in this report
NSE and NIFTY 50	The National Stock Exchange of India, and an index that tracks the combined value of India's 50 largest companies. We use the NIFTY 50 as "the market".
Return	The percentage change in a price over a period: ₹100 to ₹105 is a 5% return.
Excess return	A stock's return minus the NIFTY 50's return over the same days. Positive means it beat the market.
Trading day	A day the exchange is open; a month has about 21 and a year about 250.
Data leak	Information a model could not have had at the time of a prediction. A leak makes a model look much better than it is.
Percentile range	A forecast of a low (10th percentile), middle (50th) and high (90th) value. The real outcome should fall between low and high 80% of the time.
Ranking score (IC)	The information coefficient: the rank correlation between our ordering of the 209 stocks on a day and the order in which they actually finished. 0 means no skill; for stocks, 0.03 to 0.06 is useful.
t-statistic	How sure we can be that an average result is not luck, given how many independent test periods we have. Above about 2 is usually taken as real.
Hit rate	The share of days on which our 20 picks, as a group, grew more than a benchmark (the NIFTY 50 or the average stock) over the next month.
Walk-forward test	Train on the past, test on the next few months, move forward and repeat, as a real system would be used.
Backtest	A simulation of what the picks would have earned in the past, after trading costs.

2.1 Why stock prediction is hard

1. **The signal is tiny.** Our best signal explains only about 0.1% of the day-to-day differences between stocks; the rest is news and noise no model can foresee.

2. **There are fewer real examples than it seems.** The feature table has over 300,000 rows, but returns over overlapping months are not independent: six years contain only about 70 non-overlapping months, and the test period only 18. We therefore report ranges, not just single numbers.
3. **Direction is a coin toss.** Asked whether a single stock would go up or down, our models were right 50–52% of the time.

Our aim was therefore never to predict exact prices, but to find a small, reliable edge in the *order* of stocks and to prove that it is not the product of luck, leaks or overfitting.

3. Data Collection

We study 209 large and mid-sized NSE companies across all major sectors (banking, information technology, energy, consumer goods, pharmaceuticals, automobiles, metals and others), plus the NIFTY 50 index. Table 3 lists what we collected.

Table 3. Data collected by the project.

Data	Source	How we collected it	Amount
Daily prices: open, high, low, close, adjusted close, volume	Yahoo Finance (yfinance library)	daily_fetch.py, fetch_nifty50.py; resume from the last stored date	312,064 rows, 31 Aug 2020 to 30 Oct 2026
Company name, sector, industry	Yahoo Finance and NSE	populate_company_sectors.py	209 companies
Announcements: exact filing time, category, summary, PDF link	NSE public announcements API (JSON)	fetch_filings_metadata.py; one request per company, 1.5 s apart	169,343 announcements
Quarterly results filings (PDF)	NSE archives	download_filing_pdfs.py; newest first, resumable	2,793 PDFs, about 20 GB

Adjusted close is the closing price corrected for dividends and splits, so that a split does not look like a crash; every return in this project uses it. NSE blocks requests that do not look like a browser, so our scraper keeps a valid session with a small cookie manager, waits between requests and records its progress, so an interrupted download resumes where it stopped.

3.1 Storage

All data lives in a PostgreSQL 16 database, which lets several scripts share the data safely, enforces rules such as “one price row per stock per day”, and makes the daily update a simple insert.

Table 4. Main tables in the database.

Table	What it holds
companies	Ticker symbol, name, sector and industry of each company
market_data_daily	One row per stock per trading day: prices and volume
financial_reports, sentiment_chunks	Announcements and the FinBERT tone scores of their text
daily_sentiment_aggregates	Each company's announcement tone, summarised per day
predictions	The daily output: forecasts, the final rank of every stock and the confidence flag
prediction_accuracy, headline_hit_rates	How past predictions turned out once real prices arrived

Midway through the project the work moved to a new computer and the original database could not be recovered. Because the processed feature table had been saved as a file, we rebuilt the database from it, re-downloaded the prices and re-collected the announcements from NSE's API, which turned out to give better data (exact timestamps and categories). Since then we keep dated backups and publish the database with the code.

4. Data Cleaning

A model trained on wrong data gives wrong answers with complete confidence, so we checked the data before building any model. We found six problems, each of which would have distorted the results.

Table 5. Data problems found and how each was fixed.

Problem we found	What it would have caused	How we fixed it
Fake price rows on market holidays (the source copies the previous day)	Days with a false 0% move	Dates with zero volume across most stocks are deleted
Splits, bonus issues and demergers not adjusted (4 stocks)	Fake one-day crashes of up to 50%	Earlier prices rescaled so the series has no jump
A seven-month hole in the price history	Seven months read as one giant day	Re-downloaded; the fetcher now resumes
Weekend announcements dropped	Lost company events	Moved to the next trading day
Missing days in the NIFTY 50 data	The market treated as flat	Filled from the previous trading day
A feature that knew the future ("results week" flag)	Results far better than reality	Rebuilt from past information only

4.1 The price repair script

Cleaner/clean_prices.py runs after every price download and is safe to run repeatedly, because each repair first checks that the error is still there. It deletes **bad prints** (a single row that jumps by more than 50% and reverts the next day), deletes **fake rows** (zero-volume placeholders and whole holidays the source filled in), and rescales the history before each known **corporate action** (Table 6). Finally it lists every remaining one-day move above 20% for a person to review; genuine large moves are kept.

Table 6. Corporate actions repaired in the price history.

Stock	Date	Reason
CONCOR	1 Jan 2025	1:4 bonus issue applied on the wrong date by the data source
TATAMOTORS	14 Oct 2025	Demerger of the commercial-vehicle business
TRENT	1 Jan 2026	1:2 bonus issue applied on the wrong date by the data source
VEDL	30 Apr 2026	Demerger

4.2 Preventing leaks from the future

Leaks are the most dangerous error in this kind of project, because they make a useless model look excellent. We applied three rules:

- **Announcements count from the next trading day.** Many filings carry only a date, so we cannot tell whether they arrived before or after the market closed.
- **Features use only the past.** Every feature on day *t* uses data up to day *t*; targets look only forward.
- **A gap between training and testing** as long as the prediction horizon (Chapter 7).

Our first deep-learning model broke the second rule: a calendar feature told it in advance which weeks would bring quarterly results. Removing that leak cost the model all of its apparent advantage. It was the most important lesson of the project.

5. Processing the Data: Feature Engineering

Cleaner/build_features.py turns the clean data into one row per stock per trading day with 70 columns: identifiers, input features and the values to be predicted (Table 7).

Table 7. Feature groups built for every stock on every trading day.

Group	Example features	What they tell the model
Trend	Returns over 1 to 126 days; 12-month momentum; price against its 50-day average and 52-week high	Whether the stock has been rising or falling, and for how long
Technical indicators	RSI (14 days), MACD, stochastic RSI, Bollinger band width	Whether a recent move looks stretched
Risk	Volatility over 20 and 60 days, average true range, beta	How much the price usually swings, and how closely it follows the market
Trading activity	Volume against its 30-day average, on-balance volume, illiquidity	Unusual buying or selling interest
Market and sector	NIFTY 50 return and volatility, return relative to the sector	Whether a move is about the company or the whole market
Announcement tone	Decayed positive and negative tone, number of recent announcements	What the company recently said, and how it sounded
Calendar and identity	Day of week, month, quarter end, sector, industry	Seasonal effects and type of business

5.1 Announcement tone with FinBERT

FinBERT is a language model trained on financial text [6]; we used the version published as *FinancialBERT-Sentiment-Analysis* [7]. For each text it gives the probability of a positive, negative and neutral tone. We scored 92,165 announcement summaries and summarised them per company per day. Because news matters most on its first day, the tone decays exponentially:

$$S(t) = S(t-1) \times e^{(-\lambda)}, \quad \lambda = \ln 2 \,/\, \text{half-life}$$

The half-life is 1.5 trading days for the largest 20% of companies by traded value, 4 days for the middle 60% and 6 days for the smallest 20%, since news about large companies is absorbed faster.

5.2 Comparing, not measuring

A key decision was to describe each stock relative to the others on the same day. For the ranking models every stock feature becomes a **percentile rank** among all 209 stocks that day, from 0 (lowest) to 1 (highest). This removes differences in price level and market-wide swings, leaving only what makes a stock stand out.

5.3 What we predict

Each model's **target** is a stock's excess return over the next h trading days, in logarithmic form, where P is the stock's adjusted price and N the NIFTY 50 level:

$$\text{target}(t, h) = \ln(P(t+h) / P(t)) - \ln(N(t+h) / N(t))$$

We built targets for h = 5, 10, 21 and 126 days; the final system uses h = 21. For the ranking models the target is again converted into a daily percentile rank. The finished table has **310,540 rows and 70 columns** (31 August 2020 to 30 October 2026), stored as a Parquet file and extended automatically every day.

6. The Models We Trained

We trained models of very different kinds, from a large neural network to a one-line rule, to see what actually works on this data (Table 8).

Table 8. Overview of the models.

Model	Type	What it outputs	Role in the final system
Temporal Fusion Transformer (TFT)	Deep neural network for time series	Percentile range of the 5-day excess return	Explored in depth; not used for picks
LightGBM quantile models	Gradient-boosted decision trees	Percentile range at 5, 10, 21 and 126 days	Forecast ranges; the median is a blend member
Rank regression	Gradient-boosted trees on ranks	A score for ordering stocks	Blend member
Momentum 12-1	Rule, no training	12-month return, skipping the last month	Blend member
Per-stock history	Simple statistic	Each stock's typical past 21-day excess return	Blend member
LambdaRank, CatBoost, tuned LightGBM	Ranking and boosting variants	Scores for ordering stocks	Tested; did not pass our checks

6.1 Temporal Fusion Transformer

The TFT [1] is a neural network designed for forecasting time series. For each stock it reads the last 60 trading days: a *variable selection network* learns which inputs matter, an LSTM [13] summarises the sequence, and an *attention* layer [14] picks out the most relevant past days. It outputs three percentiles, trained with the quantile loss of Section 6.2, and one model serves all 209 stocks.

Table 9. Settings of the final TFT version (PyTorch Forecasting, NVIDIA RTX 3080 GPU).

Setting	Value	Setting	Value
Look-back window	60 trading days	Hidden size / attention heads	128 / 4
LSTM layers / dropout	2 / 0.25	Optimiser, learning rate	Adam, 0.001
Batch size	256	Training / validation data	Sep 2020–Jun 2024 / Jul 2024–Jun 2025

The first version beat a simple baseline by 0.75%, but only through the leaked "results week" feature; with the leak and data errors fixed it was 0.37% *worse* than the baseline. Investigating why, we found that *weight decay*, a penalty that pulls weights toward zero, had shrunk the variable-selection layer to about 10^{-41}, switching every input off. Without weight decay the model came back to life but memorised: training error fell 8.8% while error on new data rose 4.5% (Figure 2). Daily stock data simply holds too little repeatable pattern for a model this large.

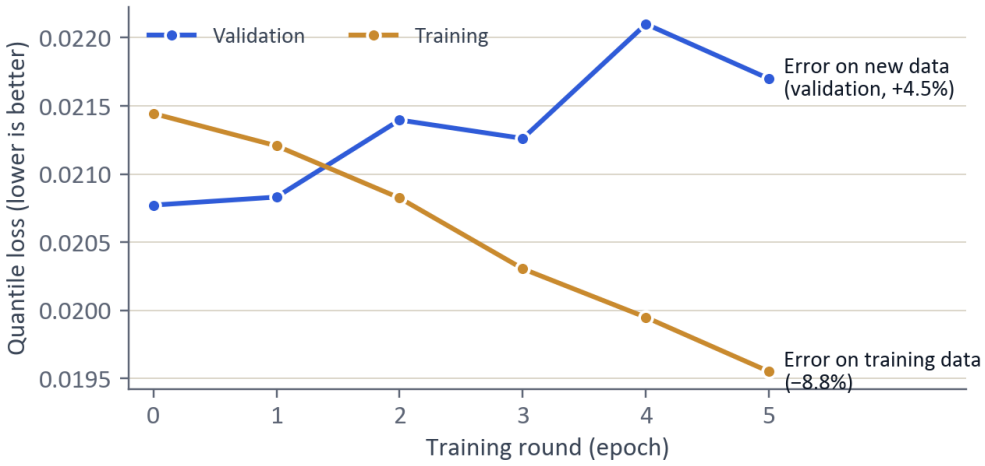


Figure 2. TFT training without weight decay: the model improves on its training data (orange) while getting worse on new data (blue), a clear sign of overfitting.

6.2 Gradient-boosted trees (LightGBM)

Gradient boosting builds hundreds of small decision trees one after another, each trained to correct the errors of the trees before it [2]. It suits tables of features like ours and trains in minutes. We trained one LightGBM model per percentile with the **quantile (pinball) loss** [12]:

$$\text{loss} = q \times (y - \hat{y}) \text{ if } y \geq \hat{y}, \quad \text{otherwise } (1 - q) \times (\hat{y} - y)$$

Here y is the outcome, $\hat{y}$ the forecast and q the percentile. For $q = 0.1$, forecasting too high costs nine times more than forecasting too low, which pushes the forecast down until about 10% of outcomes fall below it. The models use 37 inputs (29 numeric features and 8 categories), a learning rate of 0.05, 63 leaves per tree, 80% of features and rows sampled per tree, and **early stopping**: training stops once error on the last 42 trading days before the test has not improved for 50 rounds.

6.3 Rank regression

This LightGBM model learns the *order* of stocks. Its target is the daily percentile rank of each stock's future excess return, and its inputs are 33 stock features, each converted into a daily rank, plus the sector. The stock's name, the month and raw price levels are deliberately left out so it cannot memorise which stocks did well before. Its settings are cautious: learning rate 0.03, 31 leaves, at least 200 rows per leaf and an L2 penalty of 1.0.

6.4 Momentum and per-stock history

Momentum is a rule studied in finance for over thirty years [5]: stocks that rose most over the past year tend to keep doing slightly better for a while. We use the return over the last 12 months, skipping the latest month, which tends to reverse briefly:

$$\text{momentum}(t) = \ln(\, P(t - 21) \,/\, P(t - 252)\,)$$

It needs no training and so cannot overfit. **Per-stock history** is each stock's median 21-day excess return over its training data. It is weak alone, but its mistakes differ from the other members', which makes it useful in a blend.

6.5 The final model: a blend of four

1. Momentum, rank regression, the LightGBM median forecast and per-stock history each score all 209 stocks; each score becomes a percentile rank within the day (a missing score is set to 0.5).
2. The four ranks are averaged with equal weights.
3. The average is smoothed over each stock's last 5 trading days, which reduces noise and how often the picks change.
4. The 20 stocks with the highest smoothed score are the picks.

Averaging works because the members make different mistakes: their daily ranking scores are only weakly correlated (0.04 to 0.54). Weights learned from past data did worse than a plain average, because the weights themselves overfit.

6.6 The confidence flag

Each day we measure how much the four members agree, as the average Spearman rank correlation between every pair of their rankings. A day is **high confidence** when agreement is above its median over the previous 126 trading days (about six months; at least 21 days needed). We fixed this rule before measuring its effect, so it could not be tuned to the test.

6.7 What did not make it

- **LambdaRank** [10], a learning-to-rank method from web search, ranked stocks slightly backwards on the test (ranking score −0.027).
- **Automatic tuning with Optuna** [4] tried 100 settings for a LightGBM ranker. The winner scored 0.059 on its tuning data (2023 to January 2025) but 0.009 on the test: it had tuned itself to the past.
- **A stricter second tuning run**, including a **CatBoost** model [3], fixed a pass-or-fail rule in advance. Both candidates failed it, so neither was used and the test was never used to choose between them.

7. How We Train and Test

7.1 Walk-forward testing

The test period is split into six rounds of about three months (63 trading days). In every round each model is trained from scratch on all data before the round and then predicts every day of the round; the rounds move forward in time (Figure 3). Together they cover **378 trading days, from late March 2025 to September 2026**, and every model receives exactly the same test.

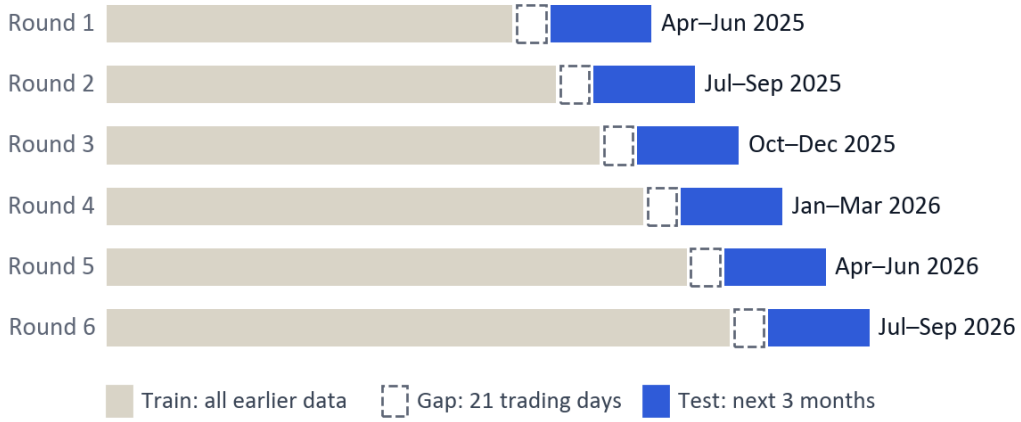


Figure 3. Walk-forward testing: train on all earlier data, leave a 21-day gap so no answer leaks in, test on the next three months.

Two details keep the test honest. A 21-day target on the last training day looks into the test period, so we leave a **gap (embargo)** as long as the horizon between training and test data [8]. And the data used for early stopping comes from the end of the training block, so the test data is never used for any decision.

7.2 How we measure success

Table 10. How results are measured.

Measure	How it is computed	Good result
Ranking score (IC)	Spearman rank correlation between our scores and the real excess returns of all stocks, each day, then averaged	0.03–0.06
t-statistic	Mean IC divided by its standard error, using only non-overlapping periods (one day in 5, or one in 21)	Above about 2
Hit rate	Share of days on which the top 20, as an equal-weight group, beat the benchmark over the next 21 days	Above the benchmark's own rate
Range accuracy	Share of outcomes inside the forecast 10th–90th percentile range	Close to 80%
Backtest return	Buy the top 20 equally, re-buy each week or month, subtract 0.18% on every trade (brokerage, tax and slippage)	Above the NIFTY 50

7.3 Rules that keep us honest

- **One test, used once.** We did not keep adjusting models until they looked good on the test months; that would turn the test into training data.
- **Rules fixed in advance** for the confidence flag and for accepting tuned models.
- **Uncertainty reported.** Hit rates come with 90% ranges from a moving-block bootstrap [11], which resamples test days in blocks of 21 so related neighbouring days stay together.

Tools. Python 3.10 (66 scripts), pandas, PostgreSQL 16, LightGBM, CatBoost, PyTorch Forecasting, Optuna, Hugging Face Transformers, Streamlit and git. Neural networks were trained on an NVIDIA RTX 3080 GPU; the tree models train in minutes on the processor.

8. The Development Journey: Eight Iterations

The final design came out of eight iterations, each answering a question raised by the one before. Three were dead ends; we keep them because each taught us something we used later.

Table 11. The eight iterations of the project.

#	What we did	What we found	Outcome
1	Trained a TFT to forecast 5-day returns	Beat a baseline by 0.75%, but only through a leaked feature	Dead end
2	Fixed the leak and the six data problems	Honest result: no better than a guess	Kept the fixes
3	Asked "which stocks will do best?" instead of "how much will each move?"	Rankings matched reality measurably (ranking score up to 0.06)	Kept
4	Tested momentum, a one-line rule	Beat every trained model: +27.4% in the backtest vs −3.8% for the NIFTY 50	Kept
5	Averaged four different models	5-day reliability rose from t = 1.4–2.2 for single models to t = 3.0	Kept
6	Diagnosed why the neural network learned nothing	Weight decay had switched off its inputs; without it, it memorised	Dead end
7	Automatic tuning with Optuna (twice)	Overfitted (0.059 → 0.009); the stricter run rejected its candidates	Dead end
8	Added the confidence flag and the live system	74% of high-confidence days beat the NIFTY 50; checked daily	Final

The turning point (iteration 3). An investor does not need to know that a stock will rise by exactly 2.3%, only which stocks are likely to do better than the others. Most of a stock's movement comes from the market as a whole, which every stock shares; predicting the *order* removes it and leaves the smaller, more stable differences between companies. The ranking score was clearly positive where direction accuracy had been stuck at 50%.

Simple beats complex, and many beat one (iterations 4 and 5). On noisy data a rule with no parameters can beat a complex model because it cannot overfit, and combining models that make different mistakes is more reliable than picking the single best one (Figure 4).

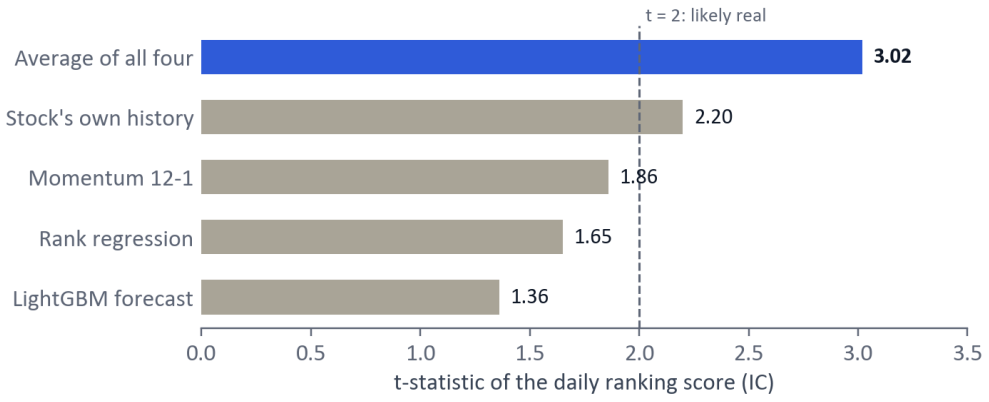


Figure 4. Reliability of the daily ranking at the 5-day horizon over the 18 test months (t-statistic of the ranking score). The plain average of four models is far more reliable than any one of them.

9. Results

All results come from the 18 test months and are reproduced by scripts in the repository (saved under results/).

9.1 Forecast ranges and ranking quality

The real outcome fell inside the LightGBM 10th–90th percentile range **83% of the time at 5 days and 80% at 21 days**, close to the 80% an honest forecast should achieve. The middle forecast, however, called a single stock's direction correctly only 50–52% of the time, which confirms that the useful information lies in ranges and rankings, not point forecasts. Table 12 compares the ranking models at 21 days.

Table 12. Ranking results at the 21-day horizon over 378 test days. Hit rates: how often the top 20 beat each benchmark over the next 21 trading days.

Model	Ranking score (IC)	t-statistic	Beat the average stock	Beat the NIFTY 50
Momentum 12-1	0.052	2.22	59.9%	66.3%
LightGBM median forecast	0.041	0.74	58.1%	67.4%
Per-stock history	0.020	0.86	44.6%	54.9%
Rank regression	0.017	0.69	41.9%	53.1%
LambdaRank	−0.027	−1.42	41.9%	49.1%
Final blend (smoothed)	0.057	1.85	65.0%	68.4%

The final blend has the highest ranking score and hit rates of any model. At 21 days the test contains only 18 non-overlapping periods, so t-statistics are lower than at 5 days even for good signals; the blend's 1.85 should be read with its 5-day value of 3.02.

9.2 Hit rates and the confidence flag

In this period most stocks happened to beat the NIFTY 50: the average of our 209 stocks did so on 63% of days. Figure 5 therefore shows both benchmarks: the NIFTY 50 (Panel A) and the stricter average stock (Panel B), where 50% means no skill.

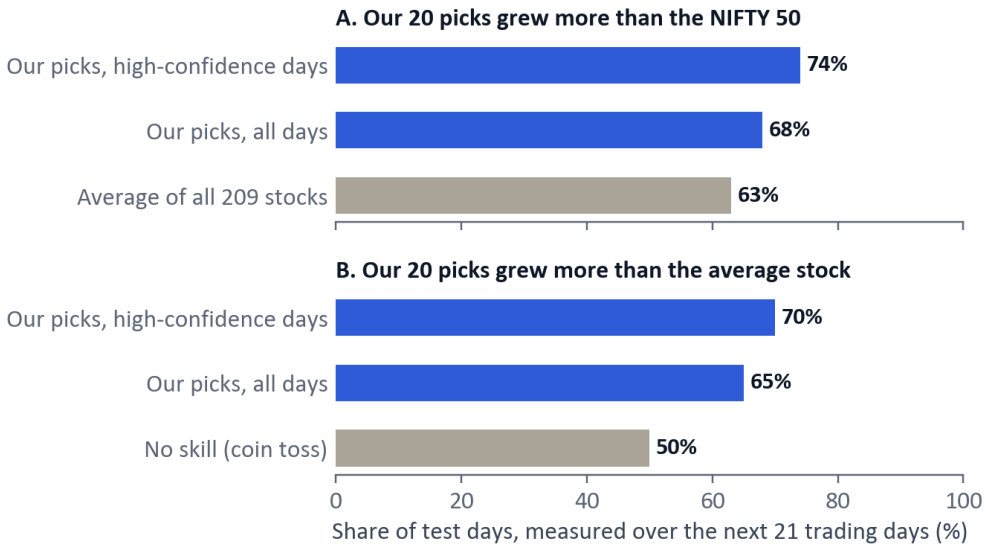


Figure 5. How often the final model's 20 picks grew more than the NIFTY 50 (A) and more than the average of all 209 stocks (B) over the next 21 trading days.

Table 13. The confidence flag on the test months, with 90% block-bootstrap ranges. The first test month is excluded because the flag needs 21 days of history.

Days	Share of days	Beat the NIFTY 50	Beat the average stock	Average excess return
High-confidence	48% (172 days)	74% (64–90%)	70% (60–84%)	+2.1% per month
Other	52% (184 days)	68% (48–83%)	66% (46–82%)	+0.8% per month
All	100% (356 days)	71% (59–83%)	68% (56–80%)	+1.4% per month

On high-confidence days the picks did better on every measure, with more than twice the average excess return. These days add up to only about eight independent months, so the true rate could lie anywhere from 64% to 90%: the flag is a guide to which days deserve more trust, not a guarantee.

9.3 Backtest

Figure 6 follows ₹100 invested at the end of March 2025 and re-invested every week in each method's top 20, after costs. Momentum grew it to ₹127 while the NIFTY 50 fell to ₹96. Buying all 209 stocks equally gave ₹113, so part of any strategy's gain in this period came simply from holding mid-sized companies. Re-bought every 21 trading days, the final blend gained **33.7%**, against 25.3% for all stocks bought equally and 5.0% for the NIFTY 50.

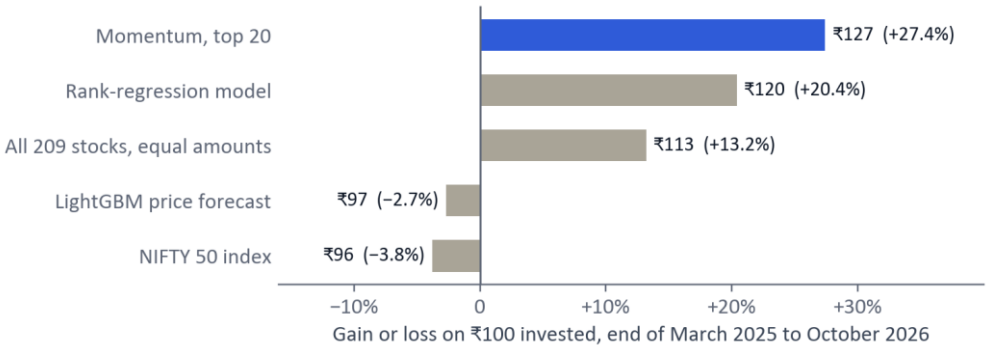


Figure 6. Backtest of ₹100 from the end of March 2025 to October 2026, re-bought every week, after a 0.18% cost on every trade.

9.4 What announcements tell us

Across 127,315 announcement events, quarterly results roughly double the size of the next two days' price move (3.8% against 1.9% on an ordinary day), but prices go up and down about equally often afterwards. FinBERT tone helps only on the day itself (+0.2% after positive summaries, −0.5% after negative ones). Announcements tell us *when* a stock will move, not *which way*, which is why they play only a small part in the final model.

10. How the System Works Day to Day

The final model is part of a system that runs by itself after every trading day:

1. **18:30 on weekdays**, Windows Task Scheduler starts run_live_tracking.py, which starts PostgreSQL if needed.
2. **Fetch and repair:** daily_fetch.py downloads the day's prices and clean_prices.py repairs them.
3. **Features:** build_features.py adds the new day to the feature table.
4. **Retrain every 15 days:** when the models are 15 or more days old, they are retrained on all data to date.
5. **Predict:** predict.py ranks all 209 stocks with the final blend, sets the confidence flag and saves the results.
6. **Drift check:** a Kolmogorov–Smirnov test warns if today's features differ markedly from the training data.
7. **Score the past:** accuracy_checker.py compares every prediction that has reached 5 or 21 trading days with what really happened.

Each run writes a dated log and takes about two minutes. A Streamlit dashboard (dashboard/app.py) reads the database and shows the results on seven pages (Table 14).

Table 14. Pages of the AlphaEdge dashboard.

Page	What it shows
Overview	Market summary and the state of the latest run
Top Picks	Today's ranking by the final model, with a banner saying whether today is a high-confidence day
Stock Deep Dive	Price history, features and forecast range for any stock
Sector Analysis	How each sector is ranked and how it has performed
News and Sentiment	Recent announcements and their FinBERT tone
Accuracy	Live tracking of how often past top-20 picks beat the NIFTY 50, beside the test-period values
Pipeline Status	When each stage last ran and whether it succeeded

Because the system scores its own predictions, testing continues beyond the 18 historical months. Live tracking started on 17 October 2026; the first 21-day results arrive in mid-November 2026, after which the Accuracy page shows live hit rates next to the research values.

Table 15. Layout of the project repository.

Folder or file	Contents
Scaper/	Data collection: NSE announcements, filing metadata, PDF downloads, cookie handling
Cleaner/	Cleaning, features, training (train_tft.py, train_lgbm.py, final_model.py), evaluation (evaluate.py, ensemble_study.py, confidence_study.py), prediction and accuracy checks
dashboard/	The Streamlit dashboard
artifacts/, results/	Trained models and the agreement history; every result table and chart in this report
run_daily_update.py	The daily orchestrator (run_live_tracking.py wraps it for the scheduler)

The code is public at **github.com/UzairManjre/stock_predict**, with the database and large data files attached as downloadable releases so every result can be reproduced.

11. Limitations and Future Work

- **Survivorship bias.** We study companies that exist today; firms that collapsed or were delisted since 2020 are missing, which flatters any stock-picking result.
- **A short test.** Eighteen months is short in markets: the 74% could really be anywhere from 64% to 90%, and the system has not been tested through a long crash.
- **A good period for mid-sized companies.** Most stocks beat the NIFTY 50 during the test, which is why we also report results against the average stock.
- **Patterns change.** From August 2024 to January 2025 every model ranked stocks backwards.
- **Not investment advice.** Real trading would also need taxes, position sizes and risk limits.

Future work. Read the 2,793 downloaded results PDFs in full with FinBERT; add delisted companies and more years of history; use the live results from November 2026 as a second, truly unseen test; and study whether the confidence flag can also decide how much to invest.

12. Conclusion

We built a complete system that collects six years of Indian market data, repairs it, turns it into 70 columns per stock per day, trains and compares several kinds of model under one honest walk-forward test, and runs by itself every trading day. The main findings are simple: predicting a stock's exact price change is close to impossible, but ranking stocks against each other is not; a plain average of four different models ranks more reliably than any single model, including a large neural network; and the models' agreement tells us which days to trust. On 18 months of unseen data the final model's 20 picks beat the NIFTY 50 over the next month on 68% of days and on 74% of high-confidence days, and beat the average stock on 65% and 70% respectively.

Just as valuable were the habits the project taught us: clean the data first, hunt for leaks, test once on unseen data, report uncertainty and keep the failures. The system is now running live, and from November 2026 it will show, day by day, whether these results hold up.

References

[1] B. Lim, S. Ö. Arık, N. Loeff and T. Pfister, "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting," International Journal of Forecasting, vol. 37, no. 4, pp. 1748–1764, 2021.

[2] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems 30, 2017.

[3] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in Advances in Neural Information Processing Systems 31, 2018.

[4] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in Proc. 25th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, 2019.

[5] N. Jegadeesh and S. Titman, "Returns to buying winners and selling losers: Implications for stock market efficiency," The Journal of Finance, vol. 48, no. 1, pp. 65–91, 1993.

[6] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," arXiv:1908.10063, 2019.

[7] A. Hazourli, "FinancialBERT-Sentiment-Analysis," model card, Hugging Face (ahmedrachid/FinancialBERT-Sentiment-Analysis), 2022.

[8] M. López de Prado, Advances in Financial Machine Learning. Hoboken, NJ: Wiley, 2018.

[9] R. C. Grinold and R. N. Kahn, Active Portfolio Management, 2nd ed. New York: McGraw-Hill, 2000.

[10] C. J. C. Burges, "From RankNet to LambdaRank to LambdaMART: An overview," Microsoft Research, Tech. Rep. MSR-TR-2010-82, 2010.

[11] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," The Annals of Statistics, vol. 17, no. 3, pp. 1217–1241, 1989.

[12] R. Koenker and G. Bassett, "Regression quantiles," Econometrica, vol. 46, no. 1, pp. 33–50, 1978.

[13] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, “Attention is all you need,” in Advances in Neural Information Processing Systems 30, 2017.

Appendix A. Running the Project

On Windows with Python 3.10 and PostgreSQL 16, from the project folder:

Get the code and data

```
git clone https://github.com/UzairManjre/stock_predict.git
python -m venv venv  &&  venv\Scripts\pip install -r requirements.txt
```

Download both releases from the repository’s Releases page, unzip the data archive into the project folder, and restore the database:

```
createdb -h localhost -p 5433 -U postgres stock_predict
pg_restore -h localhost -p 5433 -U postgres -d stock_predict --no-owner stock_predict_2026-11-01.dump
```

Reproduce the results

The walk-forward test of every model, the blends and hit rates, and the confidence flag:

```
venv\Scripts\python Cleaner\evaluate.py
venv\Scripts\python Cleaner\ensemble_study.py --horizon 21
venv\Scripts\python Cleaner\confidence_study.py
```

Run the live system

One full daily run, the 18:30 weekday schedule, and the dashboard:

```
venv\Scripts\python run_daily_update.py
powershell -File setup_live_tracking.ps1
venv\Scripts\streamlit run dashboard\app.py
```